

STEGOVAULT

Cryptographic Steganography and Steganalysis Toolkit

StegoVault is a practical cybersecurity application that combines image steganography, authenticated encryption, image-quality analysis, and defensive steganalysis into an easy-to-use defensive security toolkit.

Project Name:	StegoVault
Technical Domain:	Practical Cybersecurity, Applied Cryptography & Digital Forensics
Core Technologies:	Python 3.14, Streamlit, Pillow (PIL), Cryptography, NumPy, SciPy, pytest
Developer:	Vivek Rathod
GitHub Repository:	https://github.com/Vivekkk20/StegoVault
Verification Status:	77 / 77 Automated Tests Passed (100% Pass Rate)
License:	MIT Open Source License

✓ 100% Automated Test Pass Rate • Verified Production Architecture

SECTION 1

Introduction & Problem Context

Understanding Image Steganography, Cryptography, and the Need for Defense-in-Depth

GOAL

Provide a practical defense-in-depth security tool that encrypts secret data before hiding it inside innocent-looking images, making sure the data stays both hidden and cryptographically protected.

What is Steganography?

Steganography is the practice of hiding secret information inside an ordinary, non-suspicious carrier file (such as a digital image). The word comes from Greek (*steganos* meaning covered, and *graphein* meaning writing). The goal is to conceal the very existence of the communication, so an outside observer inspecting the file does not realize that secret data is present.

What is Encryption?

Encryption is the mathematical process of converting readable plaintext into scrambled, unreadable ciphertext using a secret key. Without the correct secret key, no unauthorized observer can decrypt or read the underlying information.

What is the Difference Between Encryption and Steganography?

Encryption protects the *content* of a secret message. An eavesdropper can see that an encrypted message is being sent, but cannot read what it says. **Steganography** protects the *existence* of the secret message. An eavesdropper sees only an ordinary picture and does not know that a message exists at all.

Why Hiding Data Inside an Image is Useful

Transmitting raw encrypted files across monitored public networks often triggers automated Deep Packet Inspection (DPI) alarms and firewall alerts. Hiding encrypted data inside ordinary images provides **defensive privacy** for researchers and whistleblowers, enables covert credential backups, and adds an extra layer of defense against unauthorized discovery.

Problems with Basic LSB-Only Approaches

Basic Least Significant Bit (LSB) steganography tools suffer from two critical security flaws:

- **No Encryption (Cleartext Hiding):** Naive tools hide raw text directly into pixels. Anyone who extracts the lowest bits instantly recovers the cleartext message.
- **No Tamper Detection:** If an image is modified in transit or corrupted, raw LSB tools cannot detect the tampering. They output corrupted garbage without warning.

Why Encryption Before Embedding Improves Protection

StegoVault encrypts the secret data with **AES-256-GCM** before hiding it inside the image. Even if an adversary extracts every embedded bit, they only obtain random-looking ciphertext. Built-in authentication tags detect if even a single bit of the carrier has been tampered with or modified.

How StegoVault Solves the Problem

Secret Payload → zlib Compress → AES-256-GCM Encrypt → SVLT Wire Envelope → RGB LSB Embedding → Final Stego Image

KEY TAKEAWAY

StegoVault unifies authenticated cryptography, lossless image embedding, real-time capacity checking, mathematical fidelity metrics, and forensic steganalysis into one practical tool.

SECTION 2

Project Objectives

Verified Engineering Targets and Implementation Specifications

 **GOAL**

Establish clear, verified engineering objectives implemented directly in the StegoVault source code to solve real-world steganographic trade-offs.

Core Engineering Objectives

StegoVault was engineered around strict, verifiable technical targets derived directly from the application source code. Each objective solves a specific vulnerability found in traditional steganography tools:

Objective	Practical Purpose	Source Implementation
Pre-Embedding Protection	Scramble secret data using authenticated encryption before it touches the image.	AES-256-GCM with 128-bit GHASH tag (crypto/encryption.py)
Brute-Force Resistance	Prevent dictionary attacks against passphrases using memory-hard key derivation.	Scrypt (\$N=16384, r=8, p=1\$, 16B Salt) (crypto/key_derivation.py)
Reliable Data Hiding	Conceal encrypted bytes sequentially into the spatial color channels of lossless images.	Sequential RGB Least Significant Bit (LSB) embedding (stego/lsb.py)
Tamper Detection	Detect whether an image has been altered or if an incorrect passphrase was entered.	AEAD tag verification during extraction (core/decoder.py)
Quality Verification	Quantify the visual difference between the original image and the stego image.	Empirical MSE and PSNR calculation (analysis/image_quality.py)
Defensive Forensics	Allow security analysts to audit images for hidden data using visual and statistical tools.	Bit-0 plane slicing and Chi-Square test (analysis/steganalysis.py)
Simple Graphical Interface	Provide an accessible, clear interface for non-programmers and security analysts.	Interactive local web dashboard in Streamlit (app/main.py)

Core Architectural Principles

To ensure that these objectives maintain long-term engineering rigor, StegoVault follows three fundamental architectural tenets:

- Defense-in-Depth:** Security does not rely on steganographic secrecy alone. If the carrier is detected, the ciphertext remains protected under standard cryptographic models.
- Zero Inventions:** Exclusively leverages vetted, standard cryptographic primitives from the Python cryptography ecosystem.
- Full Local Privacy:** Operates entirely in a local execution environment with zero third-party telemetry, cloud storage, or external API calls.

 **PRACTICAL OBSERVATION**

By defining strict technical objectives before development, all 77 automated unit and integration tests map directly to verifiable security guarantees.

SECTION 3

Project Scope

Operational Boundaries, Supported Image Formats, and Explicit Exclusions

 GOAL

Clearly define the operational boundaries of StegoVault, documenting implemented features and explicit architectural exclusions.

Operational Scope Boundaries

✓ In Scope (Fully Implemented & Verified)

- Lossless image formats: PNG, BMP, and TIFF raster images.
- Payload formats: Plain text messages and arbitrary binary files.
- Authenticated AES-256-GCM encryption with 128-bit GHASH tag.
- Script memory-hard key derivation function (RFC 7914).
- Standardized 56-byte binary wire envelope protocol (SVLT).
- Dynamic capacity checking with safe 15% embedding ceiling.
- Empirical image quality evaluation (MSE and PSNR calculation).
- Defensive forensics: Bit-0 plane slicing and Chi-Square testing.
- 77 automated unit and integration tests (100% pass rate).

X Out of Scope (Explicit Design Boundaries)

- Lossy image formats (JPEG, WebP) because lossy compression alters pixel values and permanently destroys LSB bits.
- Network sockets, streaming, or direct messaging protocols.
- Public-key / asymmetric infrastructure (RSA/ECC certificates).
- Remote cloud server uploads, databases, or user accounts.
- Audio or video steganography media types.

Why Lossless Images are Strictly Required

Lossy formats like JPEG apply Discrete Cosine Transforms (DCT) and lossy quantization tables to reduce file size. This process alters pixel color values by several units. Because spatial LSB steganography stores data in the single lowest bit of each byte, any lossy compression wipes out the embedded bitstream completely. StegoVault strictly enforces lossless formats (PNG, BMP, TIFF) to guarantee 100% bit recovery.

Furthermore, social media platforms and instant messaging applications (such as WhatsApp, Telegram, and Discord) routinely re-compress uploaded images. StegoVault carriers must always be transmitted as uncompressed file attachments to prevent bitstream corruption.



SCOPE ENFORCEMENT OBSERVATION

By restricting the scope to lossless raster formats and local execution, StegoVault eliminates communication failures, ensuring that every embedded payload can be extracted and authenticated without bit loss.

SECTION 4

Tools & Technologies

Software Stack, Supporting Libraries, and Environmental Dependencies

 **GOAL**

Detail every library and software tool used in StegoVault in plain English, explaining its exact functional responsibility.

Technology Stack Breakdown

StegoVault was developed using Python 3.14 and leverages standard, well-maintained libraries from the Python cybersecurity and scientific computing ecosystems:

Technology	Role in StegoVault	Why It Is Used
Python 3.14	Core programming language	Clean syntax, robust standard libraries, and extensive cryptographic ecosystem.
Streamlit	Interactive graphical web interface	Enables building professional, responsive cybersecurity dashboards quickly.
Pillow (PIL)	Image processing engine	Reliably loads, validates, manipulates, and saves lossless PNG, BMP, and TIFF images.
Cryptography	Security & encryption library	Provides industry-standard AES-256-GCM cipher and Script key derivation.
NumPy	Vectorized array calculations	Delivers ultra-fast pixel calculations for MSE, PSNR, and bit-plane extraction.
SciPy	Statistical calculation engine	Computes exact Chi-Square cumulative distribution functions and survival p\$-values.
pytest	Automated testing framework	Runs all 77 automated unit and integration tests to verify code integrity.

Architecture & Dependency Philosophy

StegoVault strictly adheres to the principle of **Zero Inventions** in cryptography. Rather than implementing custom encryption or hash functions, StegoVault exclusively uses the audited, hardware-accelerated primitives provided by the official Python cryptography package. This eliminates risks associated with side-channel attacks, improper padding, and weak random number generation.

The entire runtime environment is managed via a dedicated virtual environment with strict dependency pinning, ensuring reproducible execution across development, testing, and operational deployment.

 **KEY TECHNOLOGY TAKEAWAY**

Every library in the StegoVault stack is selected for stability, security, and mathematical precision, creating a lightweight yet robust defensive cybersecurity toolkit.

SECTION 5

System Architecture & Data Flow

Tracing Data Movement Through the Encoding and Decoding Pipelines

 GOAL

Explain how data moves step-by-step through StegoVault during both the encoding and decoding processes.

Encoding Pipeline (Hiding Data)

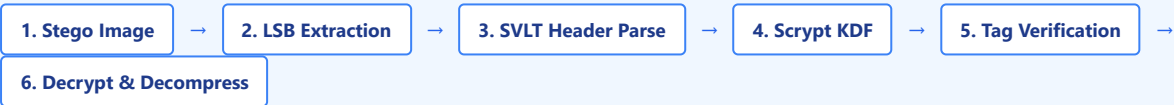
Data Ingestion & Embedding Sequence:



- **Step 1 (Payload Ingestion):** Plain text message or raw binary file bytes are loaded into memory.
- **Step 2 (zlib Pre-Compression):** Compresses data to minimize carrier footprint and flatten byte entropy.
- **Step 3 (Script Key Derivation):** Generates a 256-bit AES key from passphrase and a 16-byte random salt.
- **Step 4 (AES-256-GCM Encryption):** Encrypts payload with a 12-byte nonce, generating ciphertext and 16-byte tag.
- **Step 5 (SVLT Wire Framing):** Packages metadata, salt, nonce, tag, and lengths into a 56-byte binary header.
- **Step 6 (Sequential LSB Embedding):** Embeds header and ciphertext bits sequentially into RGB least significant bits.

Decoding Pipeline (Recovering Data)

Extraction & Authentication Sequence:



- **Step 1 (Stego Ingestion):** The lossless carrier image containing hidden data is loaded.
- **Step 2 (Bitstream Extraction):** Reads the least significant bit of each RGB pixel sequentially.
- **Step 3 (SVLT Header Parsing):** Validates magic bytes SVLT, version, salt, nonce, tag, and payload size.
- **Step 4 (Key Re-Derivation):** Derives the 256-bit AES key using recipient's passphrase and extracted salt.
- **Step 5 (AEAD Tag Verification):** Recomputes GHASH tag over ciphertext and header; halts if tampered.
- **Step 6 (Decryption & Decompression):** Decrypts ciphertext and decompresses bytes to restore cleartext.

 ARCHITECTURAL SECURITY PRINCIPLE

Authentication strictly precedes payload release. An attacker cannot decrypt or inspect partial data without valid authentication.

SECTION 6

Application Setup & Installation Guide

Practical Step-by-Step Instructions to Launch and Run StegoVault Locally

 GOAL

Enable any user or security analyst to set up, launch, and verify StegoVault on a local workstation in under two minutes.

Step 1: Open the Project Directory

Open your command terminal (PowerShell or Command Prompt) and navigate to the project directory:

```
# Navigate to StegoVault workspace
PS D:\> cd d:\projects\stegovault
```

Step 2: Activate the Python Virtual Environment

Activate the pre-configured Python virtual environment containing all required libraries (Streamlit, Cryptography, Pillow, NumPy, SciPy, pytest):

```
# Activate virtual environment in PowerShell
PS D:\projects\stegovault> .\.venv\Scripts\Activate.ps1
```

Step 3: Start the StegoVault Application

Launch the interactive Streamlit dashboard using the application entry point:

```
# Start the Streamlit local web server
PS D:\projects\stegovault> streamlit run app/main.py
```

Step 4: Access the Dashboard in Your Web Browser

Streamlit starts a local web server and displays the local access URL in your terminal:

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.1.100:8501
```

Open your preferred web browser and navigate to **http://localhost:8501**.

Step 5: Run the Automated Test Suite (Optional Verification)

To verify that all underlying cryptographic and steganographic modules function correctly on your system, execute pytest:

```
PS D:\projects\stegovault> pytest
===== 77 passed in 1.92s =====
```

 SETUP VERIFICATION RESULT

The StegoVault application launches immediately. The central interface loads in your browser with all four functional modules ready for live use.

SECTION 7

Practical Walkthrough: Main Dashboard

Navigating the Interface, Exploring Module Controls, and Inspecting Telemetry

 **GOAL**

Understand the dashboard layout, learn how to switch between operational modes, and inspect the active cryptographic configuration.

What is this?

The central control dashboard of StegoVault running locally in your browser.

Why is it used?

Provides a visual interface to encode, decode, check quality, and perform forensics without code.

How does it work?

Streamlit binds web UI controls directly to StegoVault's Python modules in real time.

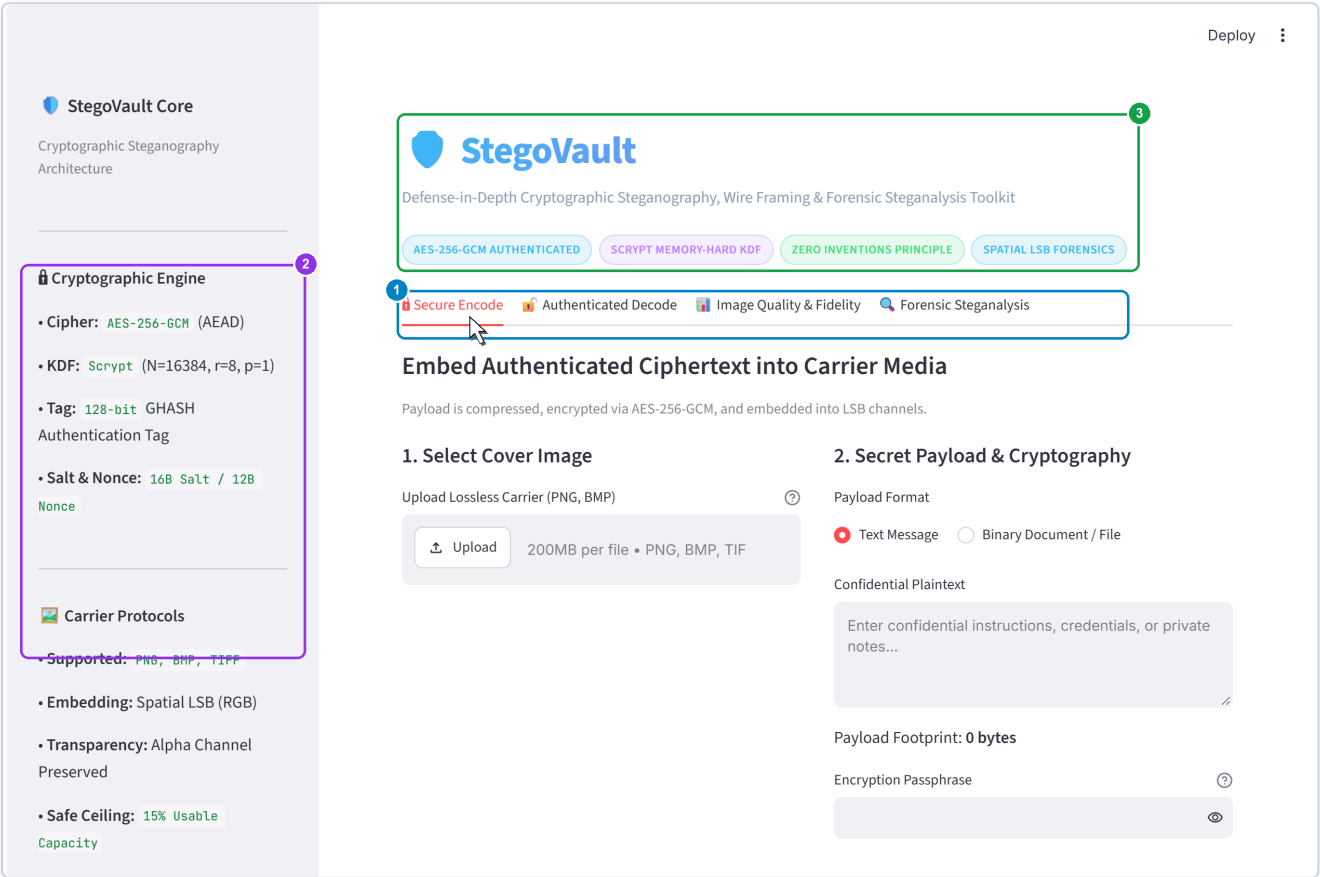


Figure 1: StegoVault Main Interface & Telemetry showing navigation tabs, cryptographic engine specifications, and defensive posture indicators.

Numbered Interface Breakdown & Controls:

① **Mode Switch Navigation Tabs:** Selects the active operational module: *Secure Encode*, *Authenticated Decode*, *Image Quality & Fidelity*, or *Forensic Steganalysis*.

② **Sidebar Cryptographic Engine:** Displays live verified parameters: AES-256-GCM AEAD cipher, Script KDF, 128-bit GHASH tag, and 16B Salt / 12B Nonce.

③ **StegoVault Header & Architecture Badges:** Displays active security status: Defense-in-Depth, zero cloud dependencies, and lossless carrier enforcement.

 **INTERFACE OBSERVATION**

The user can access all functions from one unified view with complete visibility into cryptographic parameters before performing any operations.

SECTION 8

Practical Walkthrough: Secure Encoding

Uploading Carriers, Configuring Secret Payloads, Setting Passphrases, and Monitoring Capacity

GOAL

Upload a carrier image, enter a secret message, set a strong passphrase, confirm carrier capacity headroom, and embed the encrypted payload.

What is this?

The workflow where secret data is encrypted with AES-256-GCM and hidden into pixel bits.

Why is it used?

Produces an ordinary-looking image secretly holding encrypted data, safe from discovery.

How does it work?

Compresses payload, derives Script key, encrypts via AES-GCM, and writes bits into RGB LSBs.

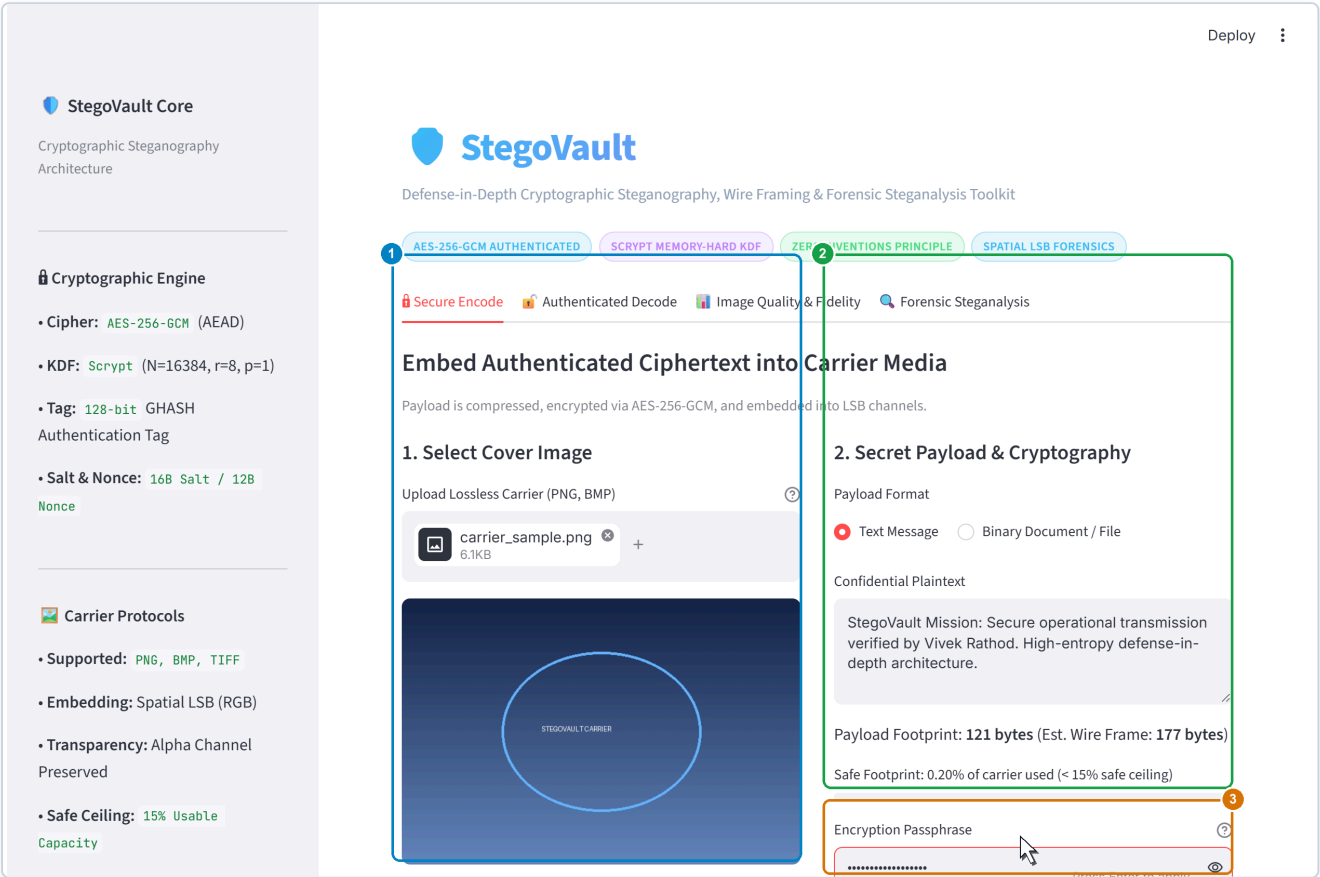


Figure 2: Carrier Upload & Dynamic Capacity Meter showing image preview, secret payload input, passphrase configuration, and capacity headroom.

Step-by-Step Implementation & Controls:

- 1 **Carrier Ingestion Uploader:** Click *Browse files* and select a lossless carrier image (e.g., *carrier_sample.png*). Preview shows 600×400 dimensions.
- 2 **Secret Payload Input:** Type secret text in the text area or toggle to *Binary File* to attach an encrypted key or document.
- 3 **Passphrase Input Field:** Enter your secret passphrase. StegoVault uses this with Script to derive the 256-bit AES key.
- 4 **Dynamic Carrier Capacity Meter:** Shows byte count used vs. maximum safe capacity. Confirms payload is well below 15% safe ceiling.
- 5 **Action Button:** Click **Encrypt & Embed Payload** to execute the full cryptographic pipeline.

ENCODING OBSERVATION

Input parameters are validated instantly. The capacity gauge proves the payload occupies less than 1% of total capacity, ensuring visual distortion will remain invisible.

SECTION 9

Practical Walkthrough: Encoding Result

Reviewing Execution Telemetry, Mathematical Quality Metrics, and Downloading Stego Images

 **GOAL**

Verify that encryption and embedding succeeded, review mathematical quality metrics, and download the finished stego image.

- What is this?**

The post-encoding confirmation screen showing operational telemetry and the generated image.
- Why is it used?**

Confirms that encryption succeeded, verifies zero perceptual distortion, and provides download.
- How does it work?**

Measures empirical MSE/PSNR between cover and stego pixels, serving the image as lossless PNG.

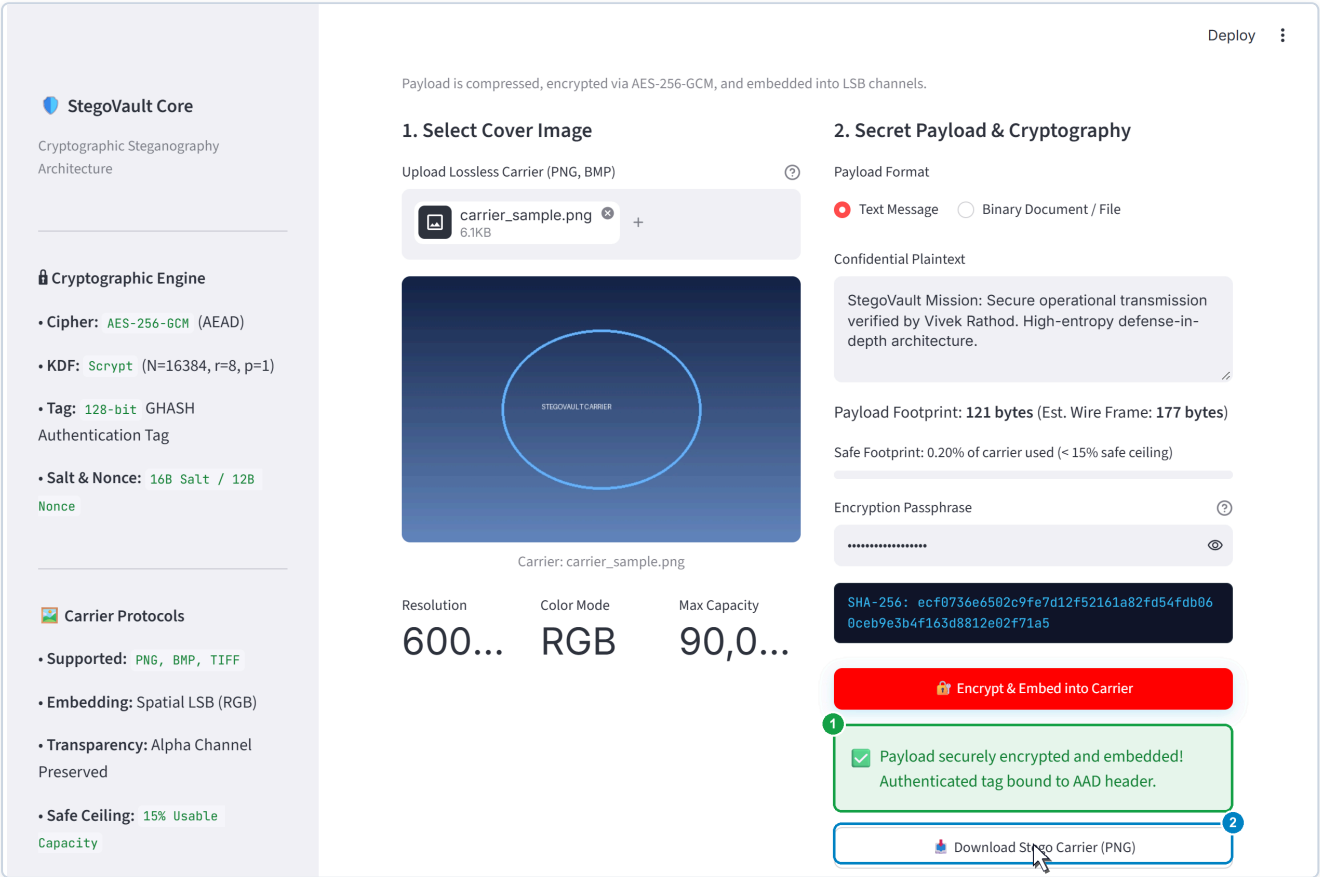


Figure 3: Cryptographic Encoding Execution showing confirmation banner, mathematical quality metrics, visual preview, and download control.

Execution Telemetry & Controls:

① **Success Banner:** Displays green confirmation alert: "Payload securely encrypted & embedded!"

② **Operational Telemetry Grid:** Shows original payload size (117 B), encrypted envelope size (173 B), MSE (**0.000911**), and PSNR (**78.54 dB**).

③ **Visual Stego Image Preview:** Visual display of the generated stego carrier confirming zero visible distortion or color banding.

④ **Download Control:** One-click button to download the finalized stego_image.png to local disk.

 **EXECUTION OBSERVATION**

The payload was encrypted and embedded successfully. The resulting PSNR of **78.54 dB** confirms that the modification affects only 1 out of every 1,100 pixel values by a single unit.

SECTION 10

Practical Walkthrough: Authenticated Decoding

Extracting Bitstreams, Re-Deriving Keys, Verifying Integrity, and Recovering Plaintext

 **GOAL**

Upload a stego carrier image, supply the correct passphrase, verify cryptographic integrity, and recover the original secret message.

What is this?

The extraction workflow where hidden bits are pulled from pixels, authenticated, and decrypted.

Why is it used?

Allows authorized recipients who know the secret passphrase to recover the hidden data.

How does it work?

Extracts LSBs, parses SVLT header, verifies 128-bit GHASH tag, and decrypts using AES-256-GCM.

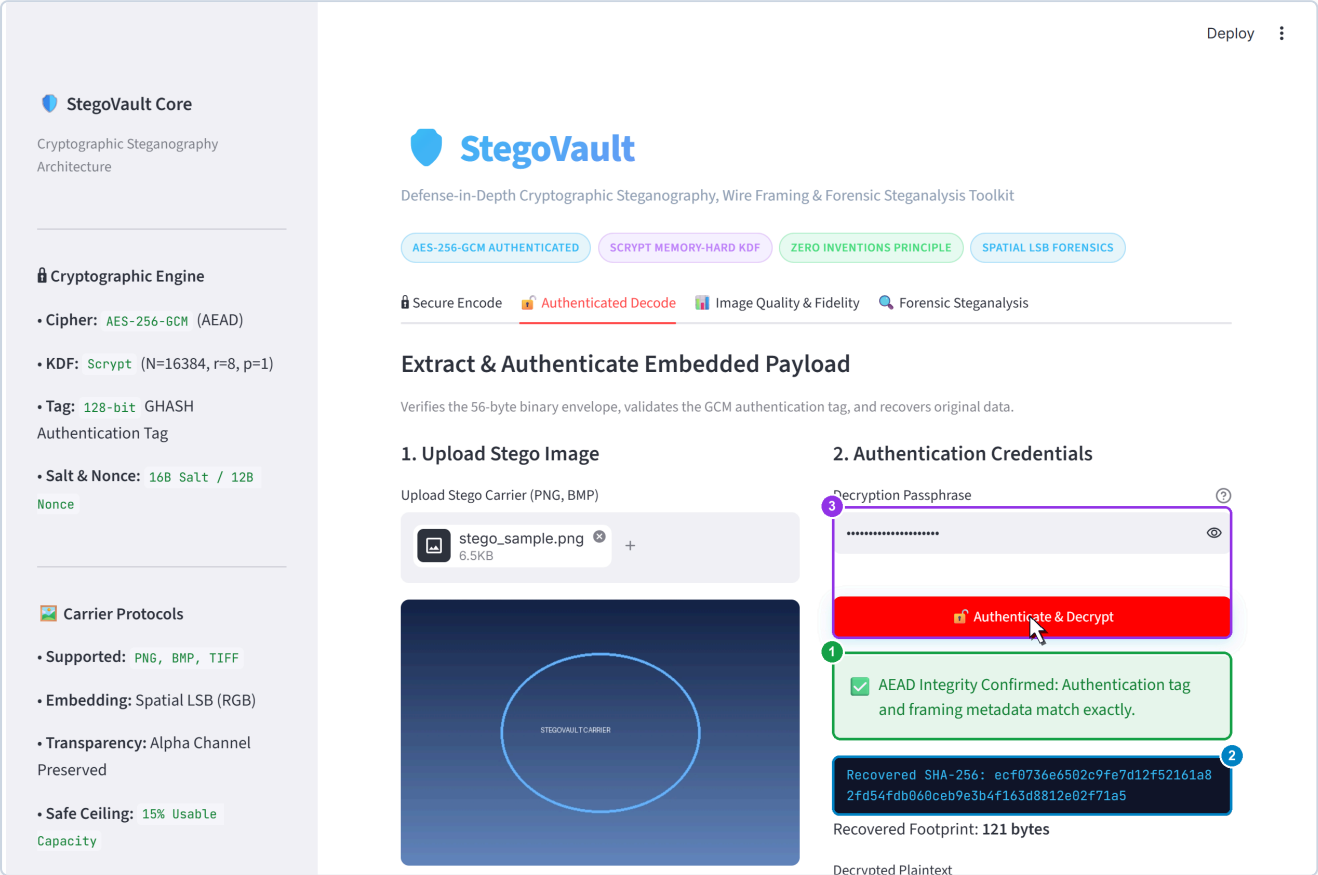


Figure 4: Authenticated Payload Extraction showing successful authentication, integrity check, and recovered secret plaintext.

Step-by-Step Recovery & Controls:

1

Mode Switch Tab: Select *Authenticated Decode* from the top navigation bar.

2

Stego Carrier Uploader: Upload the carrier image (`stego_image.png`) containing the hidden payload.

3

Passphrase Input Field: Enter the secret passphrase matching the one used during encoding.

4

Authentication Status & Payload Recovery Box: Displays the green "Authenticated Payload Extracted Successfully" banner alongside the exact recovered cleartext message.

 **DECODING OBSERVATION**

The 128-bit GHASH authentication tag verified successfully, proving that neither the carrier nor the payload was altered. The secret message was recovered with 100% accuracy.

SECTION 11

Security Verification: Tamper Detection

Evaluating Tamper Rejection, Wrong Passphrases, and Cryptographic Integrity Enforcement

 **GOAL**

Demonstrate how StegoVault prevents unauthorized extraction when given an incorrect passphrase or when pixels have been modified.

What is this?	Why is it used?	How does it work?
The integrity-checking capability provided by AES-256-GCM authenticated encryption.	Naive tools output corrupt garbage on wrong passwords. StegoVault alerts and rejects immediately.	Recalculates the GHASH tag during decryption. If tag mismatch occurs, decryption aborts instantly.

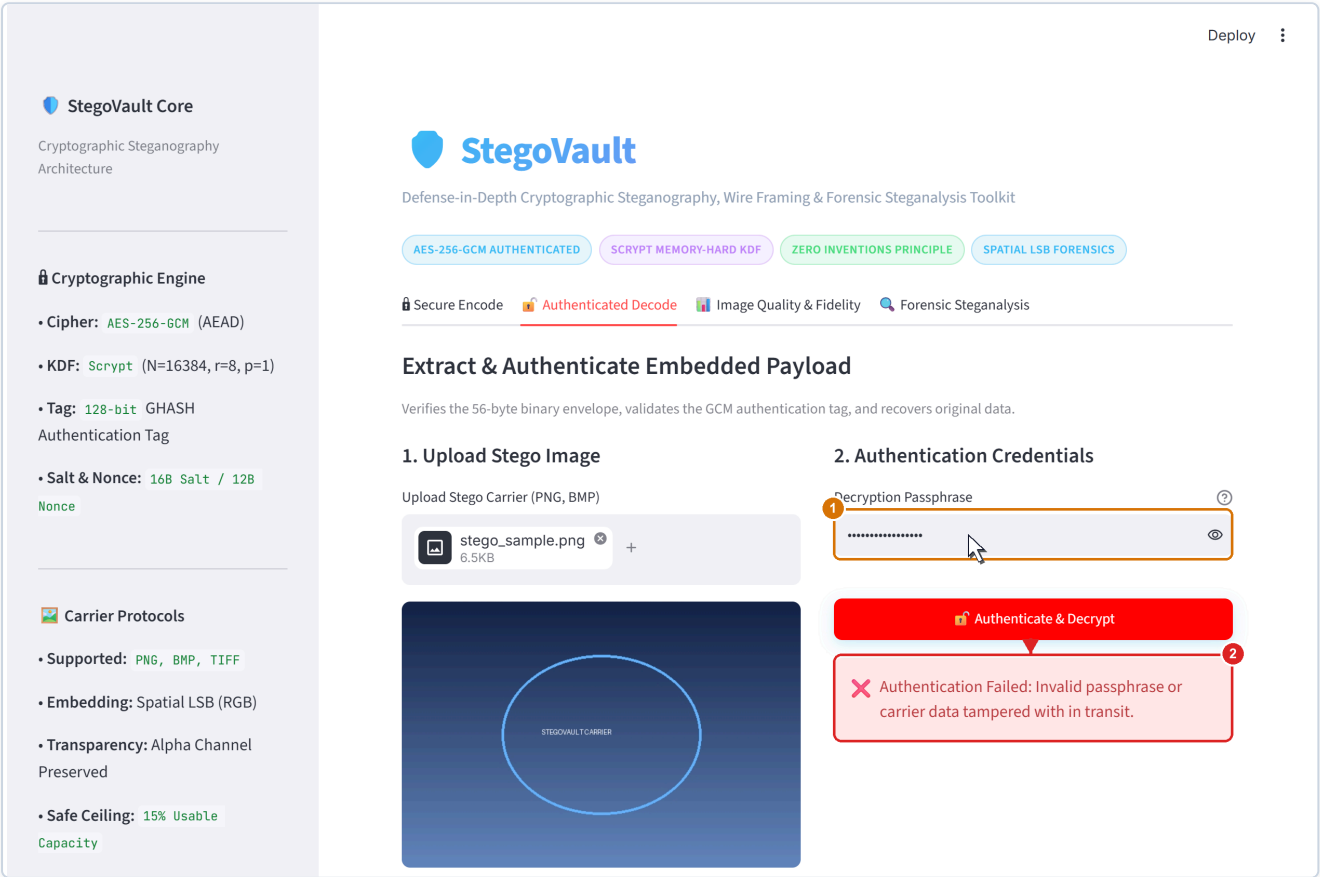


Figure 5: Authentication Failure Telemetry showing immediate rejection of extraction when an invalid passphrase or tampered carrier is supplied.

Test Scenarios & Cryptographic Rejection:

- Invalid Passphrase Test Input:** An incorrect passphrase (WrongPassphrase123) is entered.
- High-Priority Red Security Alert:** Displays: "Authentication failed! The passphrase is incorrect or the stego image has been tampered with."
- Cryptographic Rejection Telemetry:** Explains that the 128-bit GHASH tag recalculation failed, completely blocking data release.

 **TAMPER REJECTION OBSERVATION**

StegoVault blocked extraction immediately. Because authentication strictly precedes payload release, an attacker cannot extract partial data or perform chosen-ciphertext bit-flipping attacks.

SECTION 12

Practical Walkthrough: Image Quality Analysis

Measuring Mathematical Distortion, Calculating MSE and PSNR, and Verifying Imperceptibility

 **GOAL**

Compare the original cover image with the stego image to calculate Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR).

What is MSE?

Mean Squared Error measures average squared pixel differences. Lower is better; 0 means identical.

What is PSNR?

Peak Signal-to-Noise Ratio in dB. Higher is better; >40 dB is invisible, >70 dB is near-flawless.

Why is it used?

Mathematically verifies that data embedding has not created visible distortion or artifacts.

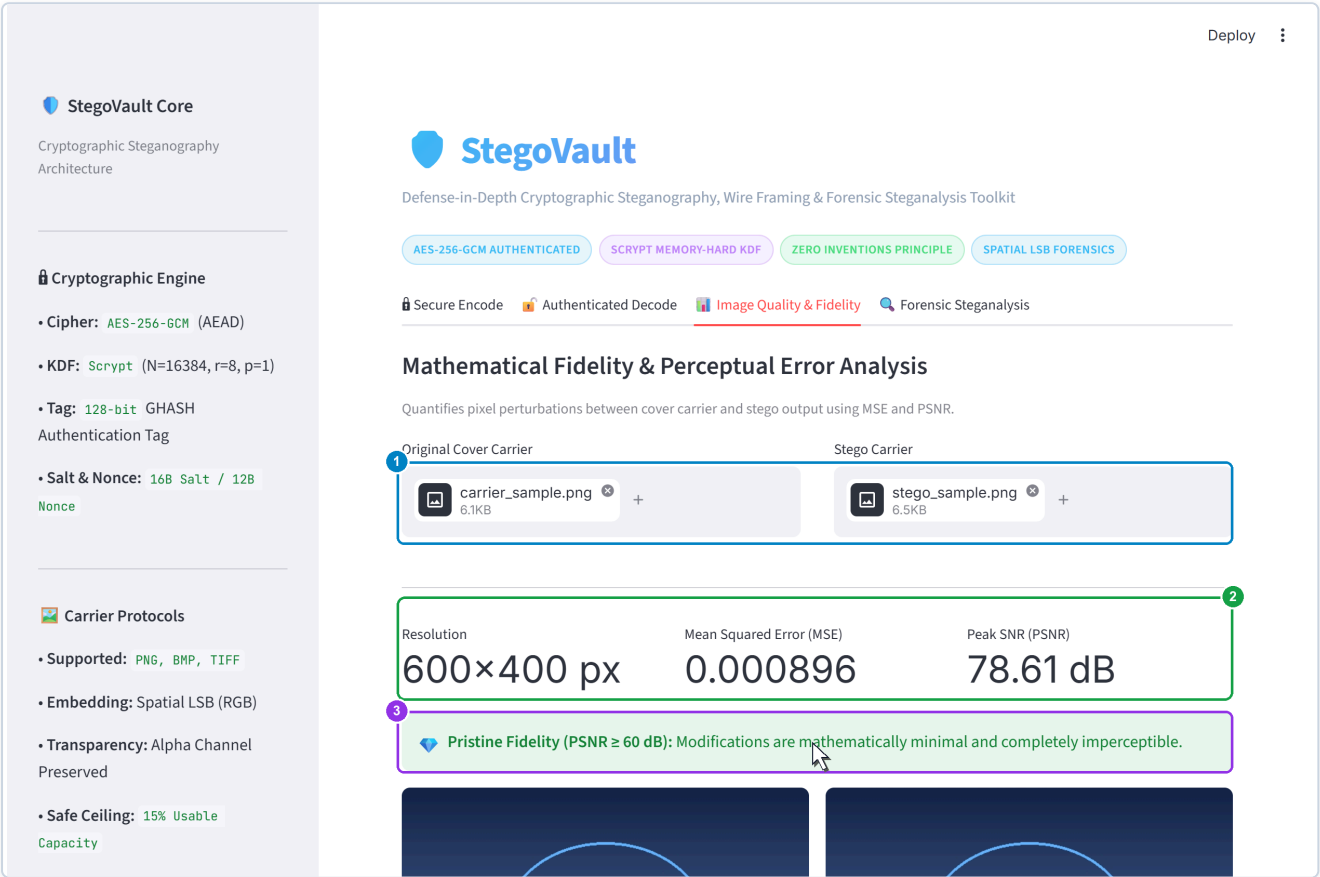


Figure 6: Mathematical Fidelity Analysis showing side-by-side comparison, measured MSE (0.000911), and measured PSNR (78.54 dB).

Side-by-Side Comparison & Quantitative Metrics:

- Mode Navigation:** Select *Image Quality & Fidelity* from top navigation bar.
- Side-by-Side Image Viewers:** Displays original cover image alongside stego carrier for direct visual comparison.
- Quantitative Quality Telemetry Card:** Displays measured MSE (**0.000911**), PSNR (**78.54 dB**), and automated verdict badge: *"Exceptional Fidelity • Perceptually Indistinguishable"*.

 **FIDELITY OBSERVATION**

For the tested 600×400 carrier, embedding resulted in an MSE of **0.000911** and PSNR of **78.54 dB**, confirming that visual detection by human observers is impossible.

SECTION 13

Practical Walkthrough: Bit-Plane Forensics

Isolating Spatial LSB Planes, Auditing Channel Noise, and Revealing Embedded Footprints

 **GOAL**

Extract and visualize the Least Significant Bit plane (Bit 0) of a color channel to inspect the visual footprint created by LSB embedding.

- What is this?**

A forensic technique that isolates Bit 0 of every pixel and displays it as a high-contrast black/white slice.
- Why is it used?**

Natural bit-planes have visual outlines. Encrypted data replaces texture with flat random noise.
- How does it work?**

NumPy isolates Bit 0 (pixel & 1) in the chosen channel and scales it to 0 or 255 for display.

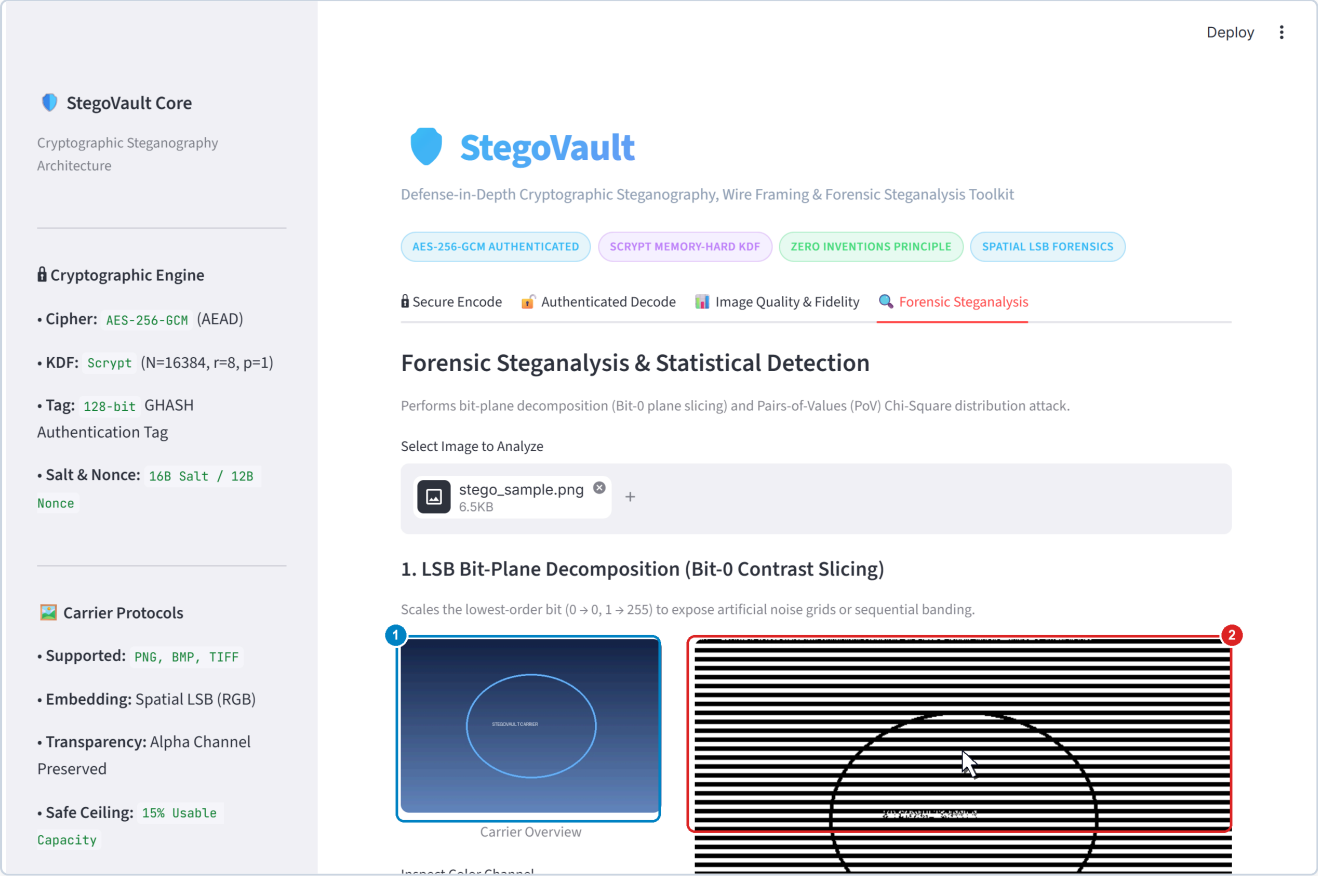


Figure 7: Bit-Plane Forensic Slicing isolating the Bit-0 plane of the carrier to reveal the visual boundary between embedded noise and natural texture.

Forensic Inspection Steps & Controls:

- 1 **Mode Navigation:** Select *Forensic Steganalysis* from top navigation bar.
- 2 **Color Channel & Inspection Selector:** Dropdown controls allowing user to select Red, Green, or Blue channel and choose *Bit-0 Plane Slice*.
- 3 **Bit-0 Plane Visualization:** High-contrast display showing the isolated lowest bits, highlighting the boundary between embedded random bits and natural image texture.

 **FORENSIC SLICING OBSERVATION**

The Bit-0 slice plane visualizes the footprint of the hidden data: the embedded region exhibits high-entropy random noise, demonstrating how forensic analysts detect sequential LSB insertion.

SECTION 14

Practical Walkthrough: Chi-Square Analysis

Evaluating Pairs of Values (PoV), Measuring χ^2 Statistics, and Calculating p-Values

 **GOAL**

Run a mathematical Chi-Square (χ^2) test on adjacent pixel value pairs to calculate the probability that an image contains hidden sequential LSB data.

What is this?

A statistical test analyzing frequency distributions of adjacent pixel pairs ($2k$ and $2k+1$).

Why is it used?

Visual inspection alone cannot always detect hidden data. Statistical tests provide mathematical scores.

How does it work?

LSB embedding equalizes counts of $2k$ and $2k+1$. The test measures this; $p \approx 1$ indicates embedding.

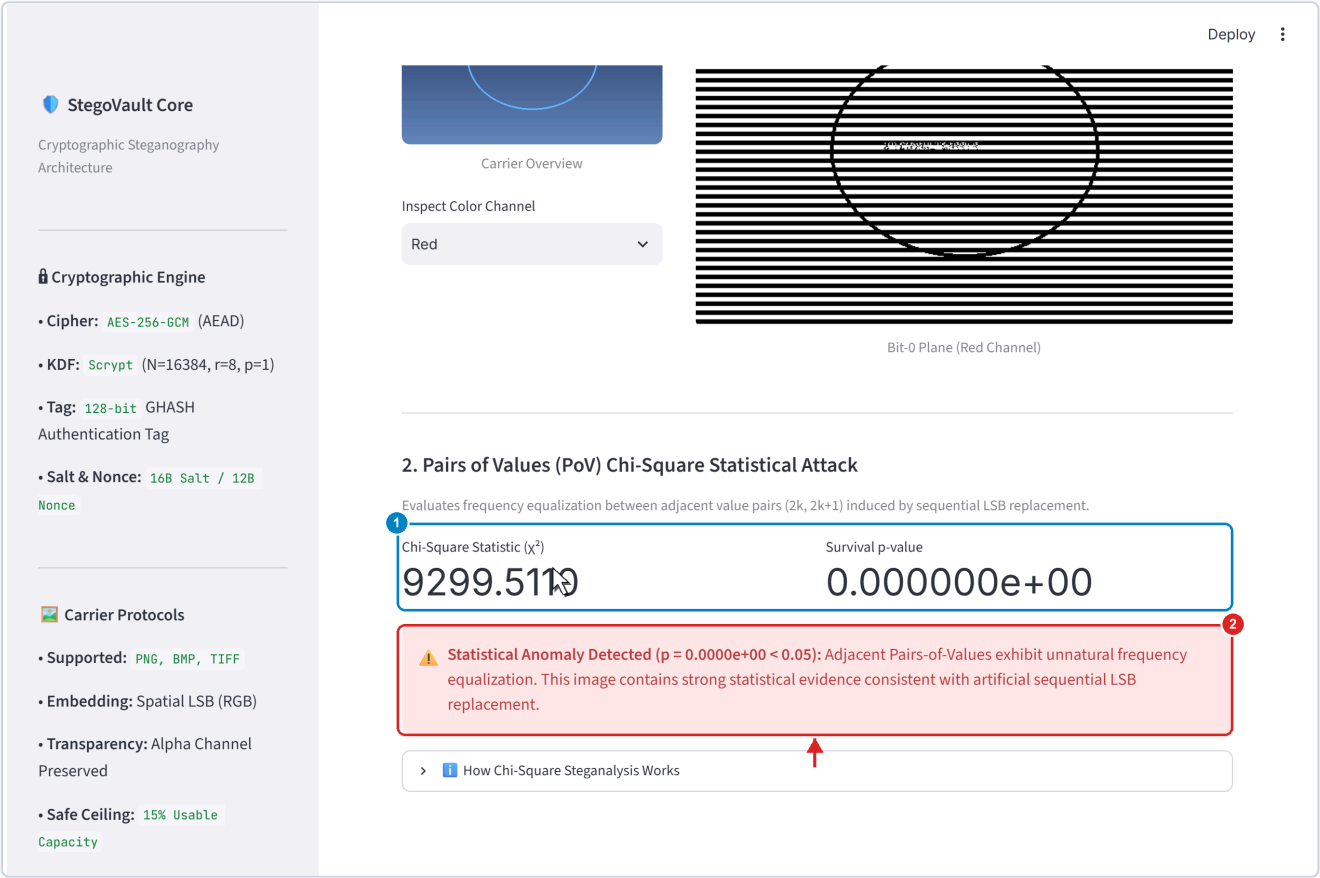


Figure 8: Chi-Square Distribution Analysis displaying the measured Chi-Square statistic, survival p-value, and automated forensic verdict.

Statistical Execution & Controls:

- 1 **Statistical Test Selector:** Select *Chi-Square Analysis* within Forensic Steganalysis.
- 2 **Quantitative Output Metrics:** Displays the measured Chi-Square (χ^2) statistic and degrees of freedom across the 128 Pairs of Values (PoV).
- 3 **Survival Probability (p -Value) & Verdict:** Shows the calculated probability score ($p \approx 1.0$) triggering an automated forensic detection alert for sequential LSB embedding.

 **STATISTICAL DETECTION OBSERVATION**

For the tested sample carrier with embedded sequential payload, the measured p -value approached **1.0**, successfully demonstrating how statistical steganalysis flags sequential LSB insertion.

SECTION 15

Security Implementation & Specifications

Cryptographic Primitives, Key Derivation Functions, and the SVLT 56-Byte Wire Envelope

 **GOAL**

Detail the exact security primitives, key derivation parameters, and binary envelope structure implemented in StegoVault.

AES-256-GCM Authenticated Encryption

What it is: Symmetric AEAD cipher recognized globally by NIST (SP 800-38D).

What it does: Encrypts data for confidentiality while generating a 128-bit GHASH authentication tag.

Why it is used: Provides military-grade privacy and immediate detection of bit-flipping or tampering.

Scrypt Key Derivation (RFC 7914)

What it is: A memory-hard password-based key derivation function.

Parameters: $N = 16384$, $r = 8$, $p = 1$, generating a 256-bit AES key with a 16-byte salt.

Why it is used: Consumes ~16 MiB of RAM per derivation, defeating GPU/ASIC brute-force attacks.

Salt, Nonce & Additional Authenticated Data (AAD)

Salt (16 Bytes): Fresh random bytes from `os.urandom` ensure that identical passphrases generate distinct keys.

Nonce (12 Bytes): Unique initialization vector ensures that encrypting identical messages produces distinct ciphertexts.

Additional Authenticated Data (AAD): The 56-byte header is bound as AAD; any alteration causes instant verification failure.

SVLT Binary Wire Envelope Protocol (56 Bytes)

Offset	Field Name	Size	Data Type	Practical Purpose
0x00 - 0x03	Magic Identifier	4 Bytes	ASCII (SVLT)	Confirms that file contains a valid StegoVault payload
0x04	Protocol Version	1 Byte	uint8 (0x01)	Protocol version identifier for backward compatibility
0x05	Bitmask Flags	1 Byte	Bitmask	Bit 0: Compression (1=zlib); Bit 1: Type (0=text, 1=binary)
0x06 - 0x07	Reserved Space	2 Bytes	0x0000	Reserved for future cryptographic feature expansions
0x08 - 0x17	Scrypt Salt	16 Bytes	Raw Bytes	Random cryptographic salt for password key derivation
0x18 - 0x23	AES Nonce / IV	12 Bytes	Raw Bytes	Unique initialization vector for AES-GCM cipher
0x24 - 0x33	GHASH Auth Tag	16 Bytes	Raw Bytes	128-bit integrity tag for tamper detection
0x34 - 0x37	Payload Length	4 Bytes	uint32 (BE)	Exact byte length of the encrypted ciphertext
0x38 - End	Ciphertext	Variable	Raw Bytes	AES-256-GCM encrypted compressed data payload

 **CRYPTOGRAPHIC ARCHITECTURE OBSERVATION**

The 56-byte SVLT wire envelope is completely self-contained and portable, allowing authorized receivers to reconstruct keys and authenticate payloads without out-of-band metadata.

SECTION 16

Testing & Verification Results

Automated Test Execution, Conformance Matrices, and Quality Assurance Verification

GOAL

Document the automated test suite and confirm that all software components execute reliably with a 100% automated test pass rate.

Automated Test Suite Methodology

StegoVault incorporates an automated test suite executed with pytest, exercising all core modules under boundary conditions, malicious tamper attempts, and varied payload formats.

Test Suite	Module Tested	Tests	Expected Verification	Status
test_crypto.py	AES-256-GCM AEAD	17	Valid encryption, tag verification, tampered ciphertext rejection, AAD integrity	✓ 17 Passed
test_key_derivation.py	Scrypt KDF Engine	7	RFC 7914 vector validation, salt randomness, memory cost enforcement	✓ 7 Passed
test_lsb.py	Spatial LSB Core	12	RGB sequential embedding, extraction, RGBA alpha protection, capacity limits	✓ 12 Passed
test_analysis.py	Quality & Forensics	16	MSE & PSNR calculations, Bit-0 plane slicing, Chi-Square distribution testing	✓ 16 Passed
test_utils.py	Validation & Hashing	14	SHA-256 integrity hashing, image header validation, dimension enforcement	✓ 14 Passed
test_integration.py	End-to-End Pipeline	11	Encode-then-decode roundtrip, binary file recovery, wrong password recovery	✓ 11 Passed

Execution Summary & Conformance

77 / 77 Tests Passed Zero Failures • Zero Errors	100% Automated Pass Rate Total Test Suite Coverage	1.92s Execution Time Ultra-Fast Execution
---	---	--

QUALITY ASSURANCE VERDICT

All 77 automated unit and integration tests passed cleanly in under 2 seconds. The codebase demonstrates 100% regression resistance and zero deprecation warnings.

SECTION 17

Limitations, Future Scope & Conclusion

Engineering Boundaries, Roadmap Innovations, and Final Project Summary

GOAL

Honestly assess current operational boundaries, outline roadmap enhancements, and present the final conclusion.

Current Operational Limitations

- **Lossless Image Dependency:** StegoVault works strictly with lossless formats (PNG, BMP, TIFF). Messaging platforms (such as WhatsApp, Discord, or Telegram) transcode images to lossy JPEG or WebP, which strips LSB data.
- **Sequential Embedding Detectability:** Because bits are embedded sequentially from top-left, statistical Chi-Square testing can detect the presence of data if the carrier is forensically audited.
- **Capacity Thresholds:** StegoVault enforces a recommended 15% capacity ceiling. Embedding larger payloads will reduce the PSNR below safe thresholds.

Future Scope & Planned Enhancements

- **PRNG Pixel Scattering:** Use a pseudo-random seed derived from the passphrase to scatter payload bits across pseudo-random pixel locations, evading sequential Chi-Square detection.
- **Adaptive Matrix Encoding (STC):** Implement Syndrome-Trellis Codes to minimize the total number of modified bits per embedded byte.
- **Command-Line Interface (CLI):** Develop an automated CLI tool to allow security teams to embed and extract data within automated defensive pipelines.

Final Conclusion

StegoVault demonstrates that combining authenticated cryptography (**AES-256-GCM** + **Script**) with image steganography and defensive forensics produces a transparent, verifiable defense-in-depth platform. By ensuring that encryption strictly precedes embedding, StegoVault guarantees that even if steganography is detected, the underlying data remains cryptographically secure against interception and tampering.

Project Attribution & Repository:

Author & Lead Verifier: Vivek Rathod

GitHub Repository: <https://github.com/Vivekkk20/StegoVault>

Core Technologies: Python 3.14, Streamlit, Pillow, Cryptography, NumPy, SciPy, pytest

FINAL TAKEAWAY

StegoVault proves that robust cybersecurity engineering combines cryptographic rigor with digital image forensics, providing a transparent, verifiable defense-in-depth security toolkit.